

Practical 2A – Memory-Mapped I/O and Hardware SPI

Starter project for EEE3096S Practical 2A. **Read this before you open the project.** It covers things the handout does not, including two places where the handout is wrong about the board.

Directory-name correction: the handout calls this folder `Practical_2A`; in the repository it is named `Practical2_A`. Use the name that appears in the repository when running the handout's `cd` command.

1. What is here

This folder **is** the project. You import `Practical2_A` itself into STM32CubeIDE:

Practical2_A/	
├─ README.md	this file
├─ Makefile	used by STM32CubeIDE to build
├─ Practical2_A Debug.launch	debug configuration
├─ STM32F051C8Tx_FLASH.ld	linker script
├─ Core/	your source code (below)
└─ Drivers/	ST HAL and CMSIS – do not edit

Each task has its own source file:

File	Task	You write
<code>Core/Inc/board_config.h</code>	—	Start here (TODO 0). Values you must look up, and board facts
<code>Core/Src/task1_gpio.c</code>	1	Memory-mapped GPIO on PC13
<code>Core/Src/task2_spi_config.c</code>	2	SPI peripheral configuration

File	Task	You write
Core/Src/task3_spi_transfer.c	3	Single-byte SPI transfer
Core/Src/task4_eeprom.c	4	EEPROM driver
Core/Src/task5_fault_case.c	5	Nothing. Do not edit. Task 5 is a diagnosis (section 6)
Core/Src/task6_fsm.c	6	Non-blocking state machine: you design the states
Core/Src/board_io.c	—	Given: LEDs and buttons
Core/Src/main.c	—	RUN_TASK selector, clock setup
Core/Inc/prac2a.h	—	Every function and shared variable

Search the project for `TODO`. The number before the dot is the task, so `TODO 2.3` belongs to Task 2. Work in order: each task builds on the one before.

2. Opening the project in STM32CubeIDE

Import it exactly as the practical brief (section 4) says:

1. `File` → `Import...` → `C/C++` → `Existing Code as Makefile Project` → **Next**
2. *Existing Code Location*: browse to the `Practical2_A` folder (the one containing this README and the `Makefile`). The project name fills in as **Practical2_A**; leave it.
3. *Toolchain for Indexer Settings*: **MCU ARM GCC** → **Finish**
4. Build with the hammer icon (`Project` → `Build Project`).
This runs the supplied `Makefile` and produces `build/Practical2_A.elf`.
5. Debug: `Run` → `Debug Configurations...` → `STM32 C/C++ Application` → **Practical2_A Debug** → **Debug**. After the first time, the bug icon runs it again.

If **Practical2_A Debug** isn't listed, create it yourself: select *STM32 C/C++ Application*, click *New*, set *C/C++ Application* to `build/Practical2_A.elf` and *Project* to `Practical2_A`, then on the *Debugger* tab set *Reset behaviour* to **Connect under reset**.

There is **no .ioc file**, deliberately. You configure the SPI peripheral yourself, from its registers. Do not create one, and do not configure SPI in CubeMX.

A fresh build shows **one defined but not used** warning in `task1_gpio.c`. That's expected, and it goes away once you finish Task 1.

3. Running one task at a time – `RUN_TASK`

At the top of `Core/Src/main.c`:

```
#define RUN_TASK    1
```

Set it to the task you're working on, build, and debug. **Only that task's program runs**, so an unfinished task can't stop the others working.

<code>RUN_TASK</code>	What the board does
1	PC13 square wave
2	SPI configured, CS held high; inspect the registers in the debugger
3	Your group byte B sent every 100 ms with CS held high
4	EEPROM: PA0 = write / read / verify; reset = read only
5	The Task 4 program with the fault case applied (section 6)
6	Your state machine: PA0 start, PA3 abort

In every mode after Task 1, PC13 keeps toggling. It's a heartbeat that shows your program hasn't hung.

4. Before you power the board

1. **Remove the P2 loopback jumpers** unless a task tells you otherwise.
PB13–PB15 go to both the EEPROM and the P2 header. A looped-back pin configured as an output can drive against another output.
 2. Connect oscilloscope ground to **board ground only**.
 3. Confirm a pin before you probe it. **PB12 (CS) is not on any header**. It only reaches pin 1 of the EEPROM.
 4. **Never write to the EEPROM in a free-running loop**. Write endurance is finite. In this project a write only happens when you press PA0.
 5. Put your scope probes on **10×** and set the channel menu to **10×** as well. If they disagree, a 3.3 V signal reads as about 33 V.
-

5. The handout is wrong about the board – read before Task 2

5.1 The SPI pin table

The handout's Task 2 table says PB13 = EEPROM serial output, PB14 = EEPROM serial input, PB15 = SCK. **That can't be right on this microcontroller**. The STM32F051 datasheet's alternate-function table fixes the SPI functions on these pins in silicon, and they can't be moved.

Measured on the board (EEPROM pin → STM32 pin):

EEPROM pin	Function	STM32 pin
1	CS#	PB12
6	SCK	PB13
2	SO (data out of the EEPROM)	PB14
5	SI (data into the EEPROM)	PB15

These pin numbers are already in `board_config.h`. **Check them yourself with a meter** before you enable SPI, and record what you find:

- board unpowered, P2 jumpers out, meter on continuity
- first anchor yourself: the EEPROM VCC pin should beep to 3V3, and VSS to GND
- if those two don't land where you expect, you're counting pins from the wrong end

Finding *which* SPI peripheral and *which* alternate function these pins use is still your job (TODO 2.2).

5.2 The EEPROM part

The handout names a **CAT25040**, which is addressed with **one** address byte (its ninth address bit goes inside the instruction). **The part fitted to the board is addressed with two address bytes, most significant first, and holds 8192 bytes.** Those values are given in `board_config.h`, because your datasheet won't tell you.

Confirm it on the bench, and write down how you did. A useful clue:

- Instructions **without** an address (write-enable, read-status) work either way.
 - Instructions **with** an address (read, write) fail if the address format is wrong.
 - A write with too few address bytes is **thrown away**, and the write-enable latch in the status register **stays set**. If you see that after a write, suspect the address format before the wiring.
-

6. Task 5 – the fault case

Set `RUN_TASK` to **5** and rebuild. The board runs the Task 4 program with **one configuration setting changed**. `RUN_TASK 4` runs the identical program without it. Same build, same output file, one number different.

This README does not tell you what the fault is. That is Task 5. Do not edit `task5_fault_case.c`.

Diagnose it from evidence, in the structure the handout requires:

1. What you expected.
2. What the board actually did.
3. The first meaningful difference.
4. The register and field responsible.
5. The correction.
6. The repeated measurement, and why it now works.

Don't hunt for it by changing SPI settings until the EEPROM responds.

- Capture the **same measurement** in both modes (`RUN_TASK 4` and 5), so the two compare directly.

- `run_task` in Live Expressions shows which mode is on the board. Check it before you trust a measurement.

7. Debugger variables

Add these to **Live Expressions** (`Window` → `Show View` → `Live Expressions` , during a debug session). You can change the values marked *writable* while the program runs, with no rebuild.

Variable	Task	Shows
<code>run_task</code>	all	which task is running
<code>task1_toggle_count</code>	1	the loop is running
<code>task1_half_period_ms</code>	1	PC13 half period in ms, <i>writable</i>
<code>dbg_spi_cr1</code> , <code>dbg_spi_cr2</code> , <code>dbg_spi_sr</code>	2	SPI register values after your configuration
<code>dbg_sck_hz_predicted</code>	2, 3	your predicted SCK frequency
<code>task3_test_byte</code>	3	the byte transmitted, <i>writable</i>
<code>eeeprom_test_addr</code> , <code>eeeprom_test_byte</code>	4, 6	address and byte to write, <i>writable</i>
<code>eeeprom_status_before</code> , <code>eeeprom_status_after</code>	4	status register around a write
<code>eeeprom_read_value</code> , <code>eeeprom_verify_ok</code>	4	read-back result
<code>eeeprom_write_wait_ms</code>	4	how long the write took
<code>eeeprom_read_loop_enable</code>	4	set to 1 to repeat a read-only transaction every 50 ms, a steady scope trace that never writes
<code>ee_state</code>	6	your current state
<code>ee_last_read</code>	6	last byte read

Variable	Task	Shows
<code>ee_use_fsm</code>	6	1 = your state machine on PA0; 0 = the Task 4 blocking path, for comparison

The `SFRs` view (`Window → Show View → SFRs`) shows the configured SPI peripheral and GPIOB register fields directly, which is useful for Task 2.

These values live in RAM. **Resetting or reflashing the board puts them all back to their defaults.**

8. Controls

Control	Function
PA0 (SW0)	Task 4: one write / read / verify. Task 6: start a transaction
PA3 (SW3)	Task 6: abort the transaction in progress
PB0–PB7	last byte read from the EEPROM
PB11 green	verification passed
PB10 red	verification failed
PC13 (header P1)	Task 1 output; heartbeat in every other mode
Reset	Tasks 4–6 start by reading the EEPROM without writing, so a stored byte survives a power-cycle